

Phase 3 Structured Notes: Complete Linear Array Queue in C

1. Current Progress Before Phase 3

Before Phase 3, you already learned two important ideas.

In **Phase 1**, you learned the basic meaning of a queue.

A queue follows this rule:

First In, First Out

This is called **FIFO**.

FIFO means:

The first value inserted is the first value removed.

You also learned that a normal queue has two important ends:

End	Meaning
front	The side where deletion happens
rear	The side where insertion happens

So in a normal queue:

Insert at rear.
Delete from front.

In **Phase 2**, you learned how to store a queue inside an array.

You compared two designs:

Design	Main Idea	Problem or Benefit
Single-pointer queue	Uses only rear	Deletion is slow because elements must shift

Design	Main Idea	Problem or Benefit
Two-pointer queue	Uses <code>front</code> and <code>rear</code>	Deletion is fast because only <code>front</code> moves

The most important Phase 2 idea was:

The two-pointer queue avoids shifting.

Instead of moving all elements after deletion, we simply move `front` forward.

Now in **Phase 3**, we take the two-pointer array queue and build a complete C program.

2. Main Goal of Phase 3

The goal of Phase 3 is to build a complete **linear array queue in C**.

A **linear array queue** means the queue is stored in a normal array from left to right.

In this phase, we use:

```
struct
malloc
functions
pointers
front and rear indexes
```

Instead of writing all logic directly inside `main()`, we organize the queue using separate functions.

The main functions are:

Function	Purpose
<code>create()</code>	Prepare the queue and allocate memory
<code>enqueue()</code>	Insert a value at the rear
<code>dequeue()</code>	Delete a value from the front
<code>display()</code>	Print all valid queue elements

Think of Phase 3 as turning the queue idea into a small reusable program.

3. What Is a Complete Linear Array Queue?

A complete linear array queue is a queue that stores its values in an array and uses two index variables:

```
front  
rear
```

The queue is called **linear** because `rear` moves forward in one direction.

Example array of size 5:

```
Index:  0   1   2   3   4  
Array: [ ] [ ] [ ] [ ] [ ]
```

If we insert 10, 20, and 30:

```
Index:  0   1   2   3   4  
Array: [10] [20] [30] [ ] [ ]
```

The logical queue is:

```
10 20 30
```

Since 10 entered first, 10 should leave first.

4. Important Conditions in Phase 3

The complete linear array queue uses these important conditions:

Condition	Meaning
<code>front == rear</code>	Queue is empty
<code>rear == size - 1</code>	Queue is full
<code>front + 1</code>	First valid element
<code>rear</code>	Last valid element

These four ideas control most of the program.

5. The Queue Structure

In C, we use a `struct` to keep related queue information together.

```
struct Queue {  
    int size;  
    int front;  
    int rear;  
    int *Q;  
};
```

This structure stores everything the queue needs.

6. Meaning of Each Structure Member

Member	Simple Meaning
<code>size</code>	Maximum number of values the queue can store
<code>front</code>	Index used for deletion
<code>rear</code>	Index used for insertion
<code>Q</code>	Pointer to the array that stores the queue values

Important point:

`front` and `rear` are index values.

They are not actual memory addresses.

Example:

```
front = 0  
rear = 2
```

This means `front` and `rear` are positions inside the array.

7. Why Do We Use a Struct?

Without a struct, we would need many separate variables:

```
int size;  
int front;  
int rear;  
int *Q;
```

That can become messy.

A struct keeps all queue-related data inside one unit:

```
struct Queue q;
```

Now `q` contains:

```
q.size  
q.front  
q.rear  
q.Q
```

This makes the program cleaner and easier to understand.

8. What Does `int *Q` Mean?

Inside the struct, we write:

```
int *Q;
```

This means:

`Q` is a pointer to an integer array.

It does not create the array immediately.

It only says:

`Q` will later point to an array in memory.

The actual array is created later using `malloc()`.

9. Why Do We Use Dynamic Memory?

We use dynamic memory because the queue size may be decided while the program is running.

For example, maybe the user wants:

```
5 spaces  
10 spaces  
100 spaces
```

If we write this:

```
int Q[5];
```

then the size is fixed in the code.

But if we use dynamic memory, we can create the array based on the chosen size.

That is why we use:

```
malloc()
```

10. What Is Heap Memory?

Heap memory is memory that the program can request while it is running.

In Phase 3, the queue array is created in heap memory.

This line creates heap memory:

```
q->Q = (int *)malloc(q->size * sizeof(int));
```

Simple meaning:

```
Create enough memory to store size integers.  
Make Q point to that memory.
```

11. The `create()` Function

The `create()` function prepares the queue before we use it.

Code:

```
void create(struct Queue *q, int size) {  
    q->size = size;  
    q->front = q->rear = -1;  
    q->Q = (int *)malloc(q->size * sizeof(int));  
}
```

The job of `create()` is to:

1. Store the queue size.
2. Set `front` and `rear` to `-1`.
3. Create the array using `malloc()`.

12. Line-by-Line Explanation of `create()`

Line 1

```
q->size = size;
```

This stores the capacity of the queue.

Example:

```
create(&q, 5);
```

Then:

```
q.size = 5
```

So the queue can store 5 values.

Line 2

```
q->front = q->rear = -1;
```

This makes the queue empty.

At the beginning:

```
front = -1  
rear = -1
```

Since both are equal:

```
front == rear
```

The queue is empty.

Line 3

```
q->Q = (int *)malloc(q->size * sizeof(int));
```

This creates the array in heap memory.

If the size is 5, this line creates space for 5 integers.

The pointer `Q` stores the starting address of that array.

Simple meaning:

```
Q now points to the array used by the queue.
```

13. Dry Run of `create()`

Suppose we write:


```
struct Queue q;  
create(&q, 5);
```

After this function runs:

```
size = 5  
front = -1  
rear = -1  
Q = memory for 5 integers
```

The queue is ready to use.

At this point, the queue is empty.

14. Why Do We Pass `&q`?

In C, if a function needs to change the original queue, we pass the queue address.

That is why we write:

```
create(&q, 5);  
enqueue(&q, 10);  
dequeue(&q);
```

The symbol `&` means:

address of

So `&q` means:

the address of q

This allows the function to change the real queue, not a copy.

15. Why Do We Use `struct Queue *q` in Functions?

Functions such as `create()`, `enqueue()`, and `dequeue()` receive a pointer to the queue.

Example:

```
void enqueue(struct Queue *q, int x)
```

This means:

q is a pointer to a Queue structure.

Because `q` is a pointer, we use the arrow operator `->`.

Example:

```
q->rear
```

This means:

```
(*q).rear
```

Simple meaning:

Go to the queue that q points to, then access rear.

16. Passing by Address vs Passing by Value

This is very important.

Passing by value

If we pass the queue normally:

```
enqueue(q, 10);
```

The function may receive only a copy.

Changes may not affect the original queue.

Passing by address

If we pass the address:

```
enqueue(&q, 10);
```

The function can change the original queue.

That is why Phase 3 uses `&q` for functions that modify the queue.

17. The `enqueue()` Function

`enqueue()` means:

Insert a value into the queue.

In a normal queue, insertion happens at the rear.

Code:

```
void enqueue(struct Queue *q, int x) {
    if (q->rear == q->size - 1) {
        printf("Queue is Full\n");
    } else {
        q->rear++;
        q->Q[q->rear] = x;
    }
}
```

18. Logic of `enqueue()`

To insert a value:

1. Check whether the queue is full.
 2. If full, print a message.
 3. If not full, move `rear` one step forward.
 4. Store the value at `Q[rear]`.
-

19. Full Condition in `enqueue()`

The full condition is:

```
q->rear == q->size - 1
```

Why?

Array indexes start from `0`.

So if the size is 5, the indexes are:

```
0 1 2 3 4
```

The last index is:

```
size - 1
```

So if:

```
rear = 4
```

there is no more space at the rear.

The queue is full.

20. Line-by-Line Explanation of `enqueue()`

Full check

```
if (q->rear == q->size - 1)
```

This checks if `rear` has reached the last index.

Full message

```
printf("Queue is Full\n");
```

This prints a warning if there is no more space.

Move rear

```
q->rear++;
```

This moves `rear` to the next position.

Store value

```
q->Q[q->rear] = x;
```

This stores the new value at the rear position.

21. Dry Run of `enqueue()`

Start:

```
size = 5  
front = -1  
rear = -1  
Array: [ ] [ ] [ ] [ ] [ ]
```

Call:

```
enqueue(&q, 10);
```

Step 1: Check full.

```
rear == size - 1?  
-1 == 4? No
```

Step 2: Move rear.

```
rear moves from -1 to 0
```

Step 3: Store value.

```
Q[0] = 10
```

Now:

```
front = -1  
rear = 0  
Array: [10] [ ] [ ] [ ] [ ]
```

The queue contains:

```
10
```

22. Another `enqueue()` Dry Run

Call:

```
enqueue(&q, 20);
```

Before insertion:

```
front = -1  
rear = 0  
Array: [10] [ ] [ ] [ ] [ ]
```

Move `rear`:

```
rear moves from 0 to 1
```

Store value:

```
Q[1] = 20
```

Now:

```
front = -1
rear = 1
Array: [10] [20] [ ] [ ] [ ]
```

The queue contains:

10 20

23. Time Complexity of `enqueue()`

`enqueue()` does not use a loop.

It only:

1. Checks if the queue is full.
2. Moves `rear`.
3. Stores the value.

So the time complexity is:

$O(1)$

$O(1)$ means constant time.

The operation does not become slower just because the queue has more elements.

24. The `dequeue()` Function

`dequeue()` means:

Delete and return the front value.

In a normal queue, deletion happens from the front.

Code:

```
int dequeue(struct Queue *q) {
    int x = -1;
```

```
if (q->front == q->rear) {  
    printf("Queue is Empty\n");  
} else {  
    q->front++;  
    x = q->Q[q->front];  
}  
  
return x;  
}
```

25. Logic of dequeue()

To delete a value:

1. Create a variable `x` and set it to `-1`.
2. Check whether the queue is empty.
3. If empty, print a message.
4. If not empty, move `front` one step forward.
5. Store the deleted value in `x`.
6. Return `x`.

26. Why Is `x` Set to `-1`?

At the beginning of `dequeue()`, we write:

```
int x = -1;
```

This means:

```
If deletion fails, return -1.
```

For example, if the queue is empty, there is no real value to remove.

So the function returns `-1` as a signal.

27. Empty Condition in dequeue()

The empty condition is:


```
q->front == q->rear
```

Why?

In this design, valid elements are between:

```
front + 1
```

and:

```
rear
```

If `front` and `rear` are equal, there is no valid element between them.

So the queue is empty.

Examples:

```
front = -1, rear = -1 → empty  
front = 2, rear = 2   → empty
```

28. Line-by-Line Explanation of `dequeue()`

Create default value

```
int x = -1;
```

This prepares a return value.

Check empty

```
if (q->front == q->rear)
```

This checks if the queue has no valid element.

Empty message

```
printf("Queue is Empty\n");
```

This prints a warning if there is nothing to delete.

Move front

```
q->front++;
```

This moves `front` to the next valid element.

Store deleted value

```
x = q->Q[q->front];
```

This stores the value being deleted.

Return deleted value

```
return x;
```

This sends the deleted value back to the caller.

29. Dry Run of `dequeue()`

Suppose the queue contains:

```
10 20 30
```

Array:

```
Index:  0    1    2    3    4  
Array: [10] [20] [30] [ ] [ ]
```

Pointers:

```
front = -1  
rear = 2
```

Call:

```
dequeue(&q);
```

Step 1: Check empty.

```
front == rear?  
-1 == 2? No
```

The queue is not empty.

Step 2: Move `front`.

```
front moves from -1 to 0
```

Step 3: Get value.

```
x = Q[0]  
x = 10
```

So `10` is returned.

Now:

```
front = 0  
rear = 2
```

The logical queue is:

```
20 30
```

30. Logical Deletion

In this array queue, deletion is usually **logical**, not physical.

That means the deleted value may still remain in the array memory, but it is no longer part of the queue.

Example before deletion:

```
front = -1  
rear = 2  
Array = [10, 20, 30]
```

After one deletion:

```
front = 0  
rear = 2  
Array may still be [10, 20, 30]
```

But the logical queue is:

```
20 30
```

Why?

Because valid queue elements are now from:

```
front + 1 = 1
```

to:

```
rear = 2
```

So the valid values are:

```
Q[1] = 20  
Q[2] = 30
```

The old value `10` is ignored.

31. Time Complexity of `dequeue()`

`dequeue()` does not shift elements.

It only:

1. Checks if the queue is empty.
2. Moves `front` forward.
3. Returns the value.

So the time complexity is:

$O(1)$

This is the main benefit of the two-pointer queue.

32. The `display()` Function

The `display()` function prints all valid queue elements.

Code:

```
void display(struct Queue q) {  
    for (int i = q.front + 1; i <= q.rear; i++) {  
        printf("%d ", q.Q[i]);  
    }  
    printf("\n");  
}
```

33. Why Does `display()` Start at `front + 1`?

This is one of the most important ideas in Phase 3.

In this design:

`front` does not point directly to the first valid element.

Instead:

`front` points before the first valid element.

So the first valid element is at:

```
front + 1
```

That is why the loop starts from:

```
int i = q.front + 1;
```

34. Why Does `display()` Stop at `rear` ?

`rear` points to the last inserted element.

So `rear` is the last valid element in the queue.

That is why the loop continues while:

```
i <= q.rear
```

So the valid display range is:

```
front + 1 to rear
```

35. Dry Run of `display()`

Suppose:

```
front = 0  
rear = 2  
Array = [10, 20, 30]
```

The display loop starts at:

```
front + 1 = 1
```

It stops at:

```
rear = 2
```

So it prints:

```
Q[1] Q[2]
20 30
```

The value 10 is not printed because front moved past it.

36. Time Complexity of display()

display() prints every valid element in the queue.

If there are n valid elements, it prints n values.

So the time complexity is:

```
O(n)
```

37. Complete Phase 3 Program

```
#include <stdio.h>
#include <stdlib.h>

struct Queue {
    int size;
    int front;
    int rear;
    int *Q;
};

void create(struct Queue *q, int size) {
    q->size = size;
    q->front = q->rear = -1;
    q->Q = (int *)malloc(q->size * sizeof(int));
}

void enqueue(struct Queue *q, int x) {
    if (q->rear == q->size - 1) {
```

```

        printf("Queue is Full\n");
    } else {
        q->rear++;
        q->Q[q->rear] = x;
    }
}

int dequeue(struct Queue *q) {
    int x = -1;

    if (q->front == q->rear) {
        printf("Queue is Empty\n");
    } else {
        q->front++;
        x = q->Q[q->front];
    }

    return x;
}

void display(struct Queue q) {
    for (int i = q.front + 1; i <= q.rear; i++) {
        printf("%d ", q.Q[i]);
    }
    printf("\n");
}

int main() {
    struct Queue q;

    create(&q, 5);

    enqueue(&q, 10);
    enqueue(&q, 20);
    enqueue(&q, 30);

    display(q);

    printf("Deleted value: %d\n", dequeue(&q));

    display(q);

    free(q.Q);

    return 0;
}

```


38. Explanation of the Complete Program

Header files

```
#include <stdio.h>
#include <stdlib.h>
```

`stdio.h` is needed for `printf()`.

`stdlib.h` is needed for `malloc()` and `free()`.

Create queue variable

```
struct Queue q;
```

This creates one queue variable named `q`.

Prepare the queue

```
create(&q, 5);
```

This creates a queue with capacity 5.

After this:

```
size = 5
front = -1
rear = -1
```

Insert values

```
enqueue(&q, 10);
enqueue(&q, 20);
enqueue(&q, 30);
```

The values are inserted in this order:

```
10, 20, 30
```

Display the queue

```
display(q);
```

This prints:

```
10 20 30
```

Delete one value

```
printf("Deleted value: %d\n", dequeue(&q));
```

This removes the front value.

Since 10 entered first, 10 leaves first.

So this prints:

```
Deleted value: 10
```

Display again

```
display(q);
```

Now the valid queue is:

```
20 30
```

So it prints:

```
20 30
```

Free heap memory

```
free(q.Q);
```

The array was created using `malloc()`.

So we release it using `free()`.

This prevents memory waste.

39. Full Program Dry Run

Starting point

```
struct Queue q;
```

A queue variable is created.

But the array is not ready yet.

After `create(&q, 5)`

```
size = 5
front = -1
rear = -1
Q = array space for 5 integers
```

Logical queue:

```
empty
```

After enqueue(&q, 10)

```
front = -1  
rear = 0  
Q[0] = 10
```

Logical queue:

10

After enqueue(&q, 20)

```
front = -1  
rear = 1  
Q[0] = 10  
Q[1] = 20
```

Logical queue:

10 20

After enqueue(&q, 30)

```
front = -1  
rear = 2  
Q[0] = 10  
Q[1] = 20  
Q[2] = 30
```

Logical queue:

10 20 30

First display(q)

Display starts from:

```
front + 1 = 0
```

Display stops at:

```
rear = 2
```

So it prints:

```
10 20 30
```

After `dequeue(&q)`

Before deletion:

```
front = -1  
rear = 2
```

Move `front`:

```
front moves from -1 to 0
```

Return:

```
Q[0] = 10
```

Deleted value:

```
10
```

Now:

```
front = 0  
rear = 2
```

Logical queue:

```
20 30
```

Second `display(q)`

Display starts from:

```
front + 1 = 1
```

Display stops at:

```
rear = 2
```

So it prints:

```
20 30
```

40. Expected Output

```
10 20 30
Deleted value: 10
20 30
```

This output proves FIFO.

The values entered in this order:

```
10 20 30
```

The first deleted value was:

```
10
```

So the first inserted value left first.

41. Visual Pointer Table

Step	Operation	front	rear	Logical Queue
1	Create queue	-1	-1	Empty
2	Enqueue 10	-1	0	10
3	Enqueue 20	-1	1	10, 20
4	Enqueue 30	-1	2	10, 20, 30
5	Dequeue	0	2	20, 30

Notice that after dequeue, `front` becomes `0`.

The value at index `0` is no longer part of the logical queue.

42. Important Rule: Valid Queue Range

In Phase 3, the valid queue elements are always from:

`front + 1`

to:

`rear`

Example:

```
front = 0
rear = 2
Array = [10, 20, 30]
```

Valid range:

```
front + 1 = 1
rear = 2
```

Valid queue:

20 30

So the value at index 0 is ignored.

43. Complexity Summary

Operation	Time Complexity	Why?
<code>create()</code>	O(1)	Sets values and allocates memory
<code>enqueue()</code>	O(1)	Moves <code>rear</code> and stores one value
<code>dequeue()</code>	O(1)	Moves <code>front</code> and returns one value
<code>display()</code>	O(n)	Prints all valid elements

Space complexity:

$O(\text{size})$

because the queue uses an array of the chosen size.

44. Overflow in Phase 3

Overflow happens when we try to insert into a full queue.

The full condition is:

```
q->rear == q->size - 1
```

Example:

```
size = 5
rear = 4
```

The queue is full because index 4 is the last valid index.

If we try to insert another value, the program prints:


```
Queue is Full
```

45. Underflow in Phase 3

Underflow happens when we try to delete from an empty queue.

The empty condition is:

```
q->front == q->rear
```

Example:

```
front = -1  
rear = -1
```

The queue is empty.

If we try to delete, the program prints:

```
Queue is Empty
```

The function returns:

```
-1
```

46. Why `display()` Passes the Queue by Value

The display function is written like this:

```
void display(struct Queue q)
```

It receives the queue by value.

That is okay because `display()` only reads the queue.

It does not need to change `front`, `rear`, or `size`.

But `enqueue()` and `dequeue()` must change the queue.

So they use:

```
struct Queue *q
```

47. Why `enqueue()` and `dequeue()` Pass by Address

`enqueue()` changes `rear`.

`dequeue()` changes `front`.

So both functions must modify the real queue.

That is why they receive:

```
struct Queue *q
```

And that is why we call them using:

```
enqueue(&q, 10);  
dequeue(&q);
```

48. Why We Must Use `free(q.Q)`

The array was created using:

```
malloc()
```

Memory created with `malloc()` should be released when we are done.

So we write:

```
free(q.Q);
```

Simple meaning:

Return the heap memory back to the system.

If we forget this, the program may waste memory.

49. Common Beginner Mistakes in Phase 3

Mistake 1: Forgetting to allocate memory

Wrong idea:

```
struct Queue q;  
q.Q[0] = 10;
```

Problem:

`Q` does not point to valid memory yet.

Correct idea:

```
create(&q, 5);
```

This prepares memory before insertion.

Mistake 2: Passing the queue by value to `enqueue()` or `dequeue()`

Wrong idea:

```
enqueue(q, 10);
```

Correct idea:

```
enqueue(&q, 10);
```

Use `&q` so the function can change the real queue.

Mistake 3: Starting display from `front`

Wrong idea:

```
for (int i = q.front; i <= q.rear; i++)
```

Problem:

`front` points before the first valid element.

Correct idea:

```
for (int i = q.front + 1; i <= q.rear; i++)
```

Mistake 4: Thinking `front` points directly to the first value

In this design, `front` does not point directly to the first valid element.

The first valid element is:

```
front + 1
```

Mistake 5: Forgetting the empty condition

Correct empty condition:

```
front == rear
```

Do not only check:

```
front == -1
```

Because after deletions, the queue can become empty when both values are equal at another index.

Example:

```
front = 2  
rear = 2
```

This is also empty.

Mistake 6: Forgetting the full condition

Correct full condition:

```
rear == size - 1
```

This is used in the simple linear array queue.

Mistake 7: Forgetting `free(q.Q)`

If memory is created using `malloc()`, release it using `free()`.

Correct:

```
free(q.Q);
```

Mistake 8: Thinking deletion physically erases the value

After `dequeue()`, the old value may still be inside the array.

But it is no longer part of the queue because `front` moved forward.

This is logical deletion.

50. Key Terms from Phase 3

Term	Simple Meaning
Linear queue	Queue stored in a normal array where <code>rear</code> moves forward
Struct	A group of related variables stored together
Dynamic memory	Memory created while the program is running

Term	Simple Meaning
Heap	Memory area used for dynamic allocation
<code>malloc()</code>	Function used to allocate memory in C
<code>free()</code>	Function used to release memory in C
<code>front</code>	Index used for deletion
<code>rear</code>	Index used for insertion
<code>Q</code>	Pointer to the queue array
<code>enqueue()</code>	Insert at rear
<code>dequeue()</code>	Delete from front
<code>display()</code>	Print valid queue values
Overflow	Trying to insert into a full queue
Underflow	Trying to delete from an empty queue
Logical deletion	Removing by moving <code>front</code> , even if the old value remains in memory

51. Phase 3 Mini Review Questions

Question 1

What is the purpose of `create()`?

Answer:

It initializes the queue and creates memory for the array.

Question 2

What does `enqueue()` do?

Answer:

It inserts a value at the rear of the queue.

Question 3

What does `dequeue()` do?

Answer:

It deletes and returns the value from the front of the queue.

Question 4

What is the empty condition?

Answer:

`front == rear`

Question 5

What is the full condition in this linear array queue?

Answer:

`rear == size - 1`

Question 6

Where is the first valid element?

Answer:

`front + 1`

Question 7

Where is the last valid element?

Answer:

rear

Question 8

Why do we pass `&q` to `enqueue()` and `dequeue()`?

Answer:

Because those functions must modify the original queue.

Question 9

Why do we use `malloc()`?

Answer:

To create the queue array dynamically in heap memory.

Question 10

Why do we use `free(q.Q)`?

Answer:

To release the memory created by `malloc()`.

52. Phase 3 Final Summary

In Phase 3, you learned how to build a complete linear array queue in C.

You used a structure:


```
struct Queue {  
    int size;  
    int front;  
    int rear;  
    int *Q;  
};
```

You learned that:

size stores the capacity.
front tracks deletion.
rear tracks insertion.
Q points to the array.

You learned the main functions:

create() prepares the queue
enqueue() inserts at rear
dequeue() deletes from front
display() prints valid elements

You learned the most important conditions:

```
front == rear    // queue is empty  
rear == size - 1 // queue is full  
front + 1        // first valid element  
rear             // last valid element
```

You also learned that deletion is logical.

That means the old value may still exist in the array, but it is ignored after `front` moves forward.

The final idea of Phase 3 is:

A complete linear array queue in C uses a struct, dynamic memory, front/rear indexes, and separate functions to perform FIFO behavior.

53. Final Takeaway

Phase 3 turns the queue theory into a working C program.

The queue still follows FIFO:

First In, First Out

But now you know how to build it using:

```
struct  
malloc  
front  
rear  
enqueue  
dequeue  
display  
free
```

The most important beginner rule is:

```
Always insert at rear.  
Always delete from front.  
Display from front + 1 to rear.
```

That is the heart of the complete linear array queue in C.